EDFIP Fineract–Odoo system architecture and IPO model

Emeraid Digital Financial Inclusion Platform
Dexta Synergy Services
Confidential — for Emeraid International Group Ltd

Purpose: Proposed architecture and Input–Process–Output model for EDFIP.
Implementation position: Apache Fineract for core banking, Odoo Community Edition as the operating platform, and a Python/FastAPI integration service.

This paper presents the Fineract–Odoo architecture, with the SaaS operating model: System Administration, configurable module packs, institution onboarding, and how a customer of an institution is held on both platforms.

---

1. Executive architecture position

EDFIP uses a composable architecture: a dedicated core banking engine and an operational platform.

Apache Fineract is the authoritative financial system of record for loans, savings, accounts, repayments, balances, financial transactions and core-ledger postings.

Odoo Community Edition provides CRM, customer and institution management, workflows, complaints, campaigns, administration, portals, operational dashboards and non-core business processes — including **System Administration** (how Emeraid onboards institutions and licences module packs).

A separate **Python/FastAPI** integration and orchestration service connects the two platforms through documented APIs and events. It manages identity mapping, retries, idempotency, status tracking, webhooks, reconciliation and failure handling.

FastAPI will not write directly to either database.

Staff work in Odoo. FastAPI is not placed in front of Odoo staff screens. It runs as a separate service in the same Emeraid environment. Mobile, USSD, payment and OEM PayGo channels use the integration layer rather than writing directly to either database. Flutter provides the field and customer Android applications.

The solution may run on the same PostgreSQL server or cluster supplied by Emeraid, but each platform will retain its own logical data store. No application will write directly into the other platform's tables.

Principle: one authoritative financial ledger, clear ownership of every data domain, API-controlled integration, and no uncontrolled dual posting.

---

2. Software as a service, System Administration, and module packs

Emeraid hosts **one** platform. Institutions are onboarded as tenants. Emeraid does not install a separate copy of EDFIP at each institution.

Practice	Meaning
One platform	A single hosted environment.

Practice	Meaning
Onboarding	An Emeraid Super Administrator creates the institution in **System Administration**.
Licensing	The institution receives a **module pack**.
Use	Staff and customers use only the licensed portion.
Suspension	Access can be locked while data is retained, and restored when the licence is current.

Each institution's data is isolated from every other institution.

2.1 System Administration

System Administration is an Odoo application used only by Emeraid Super Administrators. It is how Emeraid sells and operates EDFIP. It is not the institution's day-to-day CRM.

Role	Where they work	Responsibility
Emeraid Super Administrator	System Administration	Institutions, licences, module packs, platform connectors, preparing core banking
Institution Administrator	Institution Settings, inside their own institution	Branches, staff, branch access, local branding

Menus: Institutions · Licences · Module catalogue · System configuration · Provisioning history · Platform operators (multi-factor authentication and a full audit trail).

Institution form: identity, branding, licence, modules, organisation (Head Office and branches), first Institution Administrator, provisioning of core banking.

Provisioning does not write to Fineract tables from Odoo. Odoo asks the integration service to prepare core banking for that institution and its branches, then stores the confirmed references.

2.2 Configurable module packs

Packs are how the same platform is sold as different products. They are assembled from a catalogue (CRM, core banking, VSLA, cooperatives, Green Asset Finance / PayGo, agency banking) as a named bundle or as selected modules. A pack can be changed during the contract.

Illustrative packs

- **Microfinance** — CRM, core banking, agency, field application
- **Cooperative** — CRM, share register, core banking
- **VSLA network** — CRM, VSLA share-out, core banking
- **PayGo / green-asset** — CRM, device-linked credit, OEM token workflow, core banking

A module that is not licensed is unavailable in Odoo **and** through the API.

2.3 Bringing an institution live

1. Super Administrator creates the institution: name, type, branding, licence, module pack.
2. Head Office and branches are recorded.
3. The institution's administrator is invited (multi-factor authentication).
4. Core banking is prepared for that institution, with an office matching each branch.

5. The administrator creates staff and assigns role and branch(es). Those staff are recognised in core banking at the matching office.
6. Staff sign in to Odoo and see only their institution, licensed modules, and branches.

Branch access is enforced in Odoo, in the integration service, and in Fineract (office).

---

3. Logical architecture and boundaries

FastAPI is a dedicated Python service. It coordinates cross-platform commands and external integrations. It does not bypass Odoo or Fineract APIs and it does not write to their databases directly.

Boundary	Rule
Channels to FastAPI	Channels submit commands and queries through authenticated APIs. They do not contain authoritative financial rules.
Staff web to Odoo	Institution staff and Emeraid administrators use Odoo. FastAPI is not in front of those screens.
FastAPI to Odoo	FastAPI uses approved Odoo APIs or a documented service contract. It does not write to Odoo tables.
FastAPI to Fineract	FastAPI uses Fineract's supported REST APIs. It does not write to Fineract tables.
Odoo to Fineract	Odoo does not post financial transactions directly. Financial commands are routed through FastAPI to Fineract.
External providers	Provider callbacks enter through the FastAPI webhook receiver (signature, timestamp, replay and idempotency checks).
Database access	Separate credentials and logical stores. Cross-system joins happen in controlled reports, not by shared table access.

3.1 FastAPI responsibilities

FastAPI is not another core banking engine. It will:

- Provide a consistent EDFIP API for mobile, portal, USSD and partners
- Validate identity, institution, branch, role, licence and device context
- Translate the canonical data model into Odoo and Fineract API requests
- Keep stable external IDs for customers, accounts, loans, assets, devices and transactions
- Protect every financial command with an idempotency key
- Coordinate workflows that span Odoo, Fineract and third-party providers
- Store command state so interrupted requests can be resumed
- Receive and verify provider webhooks
- Publish and consume integration events
- Maintain reconciliation and exception queues
- Expose versioned OpenAPI documentation

FastAPI will not duplicate Fineract's loan or savings calculations. Fineract remains the authority for balances, schedules, postings and ledger results.

---

4. Major components

4.1 Channel layer

- Staff web application (Odoo)
- Responsive customer portal
- Customer Android application
- Field and agent mobile application with offline capability
- USSD flows, subject to provider readiness
- Partner and integration APIs

Channels do not contain authoritative financial rules. They collect user actions, display permitted information and submit commands.

4.2 Identity and access management

An OpenID Connect identity provider will provide a consistent identity model across Odoo, Fineract, APIs and mobile services, including:

- Multi-factor authentication for administrators, finance users and approval roles
- Role, institution, branch and portfolio (account officer) scope
- Service identities for system-to-system calls
- Short-lived access tokens
- Device registration and revocation for field applications
- Maker-checker and segregation of duties for sensitive operations

The exact provider will be confirmed during inception and recorded in the software bill of materials.

4.3 Odoo operational platform

Odoo will host or support:

- Multi-tenant administration, System Administration, licences and module packs
- Customer, member and institution records, including home branch and account officer
- KYC workflow and document management
- Full CRM: leads, prospects, visits, tasks, complaints, campaigns and segmentation
- Cooperative and VSLA operational workflows
- Green-asset registry, installation and customer-service workflows
- OEM PayGo connector orchestration and token delivery workflow
- Agency and field operations that do not require authoritative financial posting
- Customer portal and operational dashboards
- Reporting views sourced from confirmed financial data

Odoo may display financial information. Loan balances, repayment status and ledger values must come from Fineract or from a clearly labelled, reconciled read model derived from Fineract.

4.4 Apache Fineract core

Fineract will own:

- Financial products
- Client financial accounts and loan accounts
- Savings and deposit accounts
- Loan schedules and repayment allocation
- Interest, fees and penalties supported by the approved configuration
- Repayment and disbursement transactions
- Financial balances and transaction history
- General-ledger and accounting postings
- Financial audit and transaction references

Any EDFIP-specific financial requirement not provided by the selected Fineract release will be implemented as a controlled Fineract extension, a surrounding financial service, or a documented approved workflow. No gap should be hidden inside Odoo merely to make the architecture appear complete.

4.5 Integration and orchestration layer

The integration layer is the control point between Odoo and Fineract. It will provide canonical identifiers, Odoo–Fineract mappings, authentication, idempotency keys, outbox/inbox processing, retry with backoff, dead-letter and recovery queues, webhook verification, correlation IDs, reconciliation, versioned API contracts and integration tests.

It is implemented as a separate Python/FastAPI service in the same environment as Odoo.

4.6 Data and reporting layer

Financial data will be read from Fineract through approved APIs or controlled read-only replication. Odoo will maintain operational read models where required for CRM and workflow performance.

Reports will identify their source and freshness:

- **Financial authoritative** — directly confirmed by Fineract
- **Operational** — calculated from Odoo-owned operational data
- **Combined** — produced by the reporting layer after joining approved, reconciled identifiers — not by direct cross-application table writes

---

5. Customers of an institution

EDFIP is not only a staff system. Each institution serves customers — the people who hold savings, loans and, where licensed, PayGo devices. The pattern is the same as a bank: the person belongs to the **institution**, is served from a **home branch**, and is assigned an **account officer**. Accounts sit under that person.

	Odoo (operating platform)	Fineract (core banking)
Customer record	Master: name, contacts, KYC, documents, CRM, complaints, home branch, account officer	Matching financial client so accounts can exist
Accounts	Staff and the customer **see** confirmed balances	Master: savings, loans, schedule, balance, ledger
Customer app and portal	Sign-in and profile	That customer's own accounts; repayment instructions

The integration layer keeps one stable customer ID on both sides. There is not a second, disconnected customer book.

The account officer sees their portfolio. Other branches of the same institution do not see that customer unless authorised. Other institutions never do. On the customer application or portal, the person sees only their own record and accounts.

---

6. Data ownership and consistency

6.1 System of record

Data domain	Authoritative system	Other system's copy	Consistency rule
Tenant identity, licence, module pack	Odoo	Fineract tenant mapping	Odoo creates and maps the tenant before financial use
Branch structure	Odoo	Fineract offices	Each branch maps to a Fineract office
Customer identity, CRM, home branch, account officer	Odoo	Fineract client reference	Stable external customer ID; no duplicate client creation
KYC workflow and documents	Odoo	Fineract KYC status where required	Odoo owns evidence; Fineract receives approved status
Loan products and financial parameters	Fineract	Odoo read-only projection	Fineract controls the version used for calculation
Loan accounts and schedules	Fineract	Odoo read-only projection	Fineract is authoritative
Savings accounts and balances	Fineract	Odoo read-only projection	Fineract is authoritative
Repayments, disbursements and reversals	Fineract	Odoo transaction reference/status	Only Fineract may post or reverse financial entries
General ledger	Fineract	Odoo reporting projection	No parallel ledger in Odoo
VSLA meetings and operational events	Odoo	Financial postings sent to Fineract	Meeting remains in Odoo; money is posted to Fineract

Data domain	Authoritative system	Other system's copy	Consistency rule
Green assets and installation records	Odoo	Fineract asset/loan reference	Asset state is Odoo-owned; loan and payment state is Fineract-owned
PayGo token workflow	Odoo / integration layer	Fineract repayment/ eligibility	Token issuance requires confirmed financial eligibility
Complaints, visits and campaigns	Odoo	Optional summary only	Odoo is authoritative
Audit correlation	Both, linked by correlation ID	Cross-reference only	No deletion or alteration of financial audit history

6.2 Consistency model

The architecture will not attempt an unsafe distributed transaction across Odoo and Fineract. Instead:

1. A financial command is accepted by the integration layer.
2. The command is assigned a unique idempotency key.
3. Fineract validates and posts the financial transaction.
4. The integration layer stores the Fineract transaction ID and confirmed status.
5. Odoo updates its read model from the confirmed result or event.
6. If the response is interrupted, the command remains pending and is retried with the same key.
7. A reconciliation job identifies any unmatched, delayed or conflicting records.

This prevents double posting while making failures visible instead of silently overwriting data.

---

7. Critical end-to-end flows

7.1 Customer onboarding

Input: customer details, KYC documents, consent, institution, branch, account officer

→ Odoo validates identity, duplicates, KYC workflow and approvals; records home branch and officer
→ Integration layer assigns a stable external customer ID
→ Fineract creates the financial client at the matching office, with the officer reflected for portfolio
→ Output: approved customer, linked Odoo/Fineract IDs, KYC status, branch and officer

7.2 Loan origination and approval

Input: application, product, amount, tenor, cash-flow data, guarantors, documents, branch

→ Odoo captures CRM, documents and appraisal workflow
→ Integration layer retrieves approved financial product rules
→ Fineract creates and processes the loan account at the customer's branch

→ Odoo records workflow status and displays the confirmed account reference
→ Output: approved or rejected loan, schedule, account ID and audit trail

7.3 Repayment and reconciliation

Input: repayment from teller, agent, gateway, USSD, customer app or field device

→ Channel validates user, device, limits and idempotency key
→ Integration layer sends one financial command to Fineract
→ Fineract posts repayment and updates balance/ledger
→ Event/response updates Odoo and triggers receipt/notification (staff and, where appropriate, customer)
→ Output: confirmed transaction, updated balance, receipt and reconciliation record

7.4 Green Asset Finance and PayGo

Input: customer, asset, device ID, loan and repayment event

→ Odoo records asset, installation and OEM relationship
→ Fineract confirms loan, repayment and eligibility state
→ Integration layer calls OEM connector with idempotency and retry protection
→ Odoo records token state and sends approved delivery notification
→ Output: token, delivery status, device state, audit trail and exception record

7.5 Offline field repayment

Input: offline repayment event, device ID, user ID, timestamp and unique event ID

→ Mobile app stores encrypted event in the local outbox
→ Sync service uploads the event when connectivity returns
→ Server rejects duplicates by event/idempotency key
→ Integration layer posts only accepted financial commands to Fineract
→ Odoo receives confirmed status and updates field/CRM views
→ Output: accepted, rejected or pending transaction with no silent loss

7.6 Institution onboarding

Input: institution profile, module pack, branding, Head Office and branches, administrator

→ Odoo System Administration creates the institution and organisation
→ Integration layer prepares the Fineract tenant and matching offices
→ Administrator assigns staff to roles and branches
→ Output: live institution, licensed modules, staff limited to their branches

---

8. IPO model

IPO means Input–Process–Output. It describes what enters EDFIP, what the platform does with it, and what result it produces.

8.1 Platform-level IPO

Inputs	Processes	Outputs
Customer and institution data	Validate, deduplicate, approve and map identity	Approved client and institution records
KYC documents and consent		

Inputs	Processes	Outputs
	KYC workflow, verification, retention and access control	KYC status, evidence record and alerts
Product configuration	Validate product rules and publish approved version to Fineract	Active loan/savings product
Loan application and appraisal data	Credit workflow, approval limits and Fineract account creation	Loan decision, schedule and account reference
Deposits, repayments and disbursements	Authenticate, validate limits, post through Fineract and reconcile	Confirmed transaction, balance, receipt and GL entry
VSLA meeting records	Capture attendance, shares, fines, social fund and loan events	Meeting record, group totals and financial commands
Asset and device details	Link customer, asset, loan, device and OEM	Asset record and PayGo eligibility state
Repayment eligibility event	Call OEM connector, retry safely and record outcome	Token, delivery status and audit trail
Offline mobile events	Encrypt, queue, deduplicate, sync and apply conflict rules	Accepted, rejected or pending field events
Payment gateway callbacks	Verify signature, deduplicate and match payment reference	Posted payment or unmatched-payment case
User actions and approvals	Enforce RBAC, maker-checker, segregation of duties and audit	Approved/rejected action and audit event
Operational and financial records	Aggregate, filter, reconcile and apply institution permissions	Reports, dashboards, donor outputs and alerts

8.2 Core business-process IPO

Business process	Input	Processing	Output
Tenant onboarding	Tenant profile, modules, branding, users	Odoo configures tenant; integration creates Fineract mapping	Active tenant with controlled entitlements
Customer onboarding	Bio-data, IDs, documents, consent, branch, officer	Odoo KYC and duplicate checks; Fineract client link	Approved customer with linked IDs, branch and officer
Savings account	Customer, product, opening amount	Product validation and Fineract account creation	Active savings account and balance
Loan application	Customer, loan product, appraisal and approvals	Odoo workflow; Fineract loan creation and schedule	Approved loan and repayment schedule
Repayment	Amount, account, channel and event ID	Fineract posting with idempotency and ledger update	Confirmed repayment and receipt

Business process	Input	Processing	Output
Teller transaction	Teller, till, customer and cash amount	Limits, maker-checker, Fineract posting and till update	Receipt, balance and till audit
VSLA meeting	Attendance, shares, fines, social fund and cash count	Offline-capable event capture and financial posting	Meeting result, discrepancy case and group report
PayGo token	Loan, asset, device, repayment status	Fineract eligibility check; OEM request; delivery tracking	Token or controlled exception case
Agent collection	Agent, customer, repayment and location	Device/user validation, limits, Fineract posting, commission	Customer receipt and agent settlement entry
Migration	Source files, mappings and opening balances	Validate, preview, approve, import and reconcile	Migrated records and signed reconciliation
Reporting	Odoo operational data and Fineract financial data	Controlled extraction, mapping and aggregation	Institution, group, financial and donor reports

---

9. Security architecture

Security is applied beyond server hardening:

- OpenID Connect / OAuth2, multi-factor authentication and least-privilege access
- Institution, branch, licence and portfolio scope enforced at the application and service layer
- No direct database access from mobile, partners or Odoo screens for financial posting
- TLS for all service calls and signed webhooks
- Encrypted secrets and API credentials with rotation
- Field-level encryption for sensitive identity data and PayGo credentials
- Idempotency and replay protection for financial commands and webhooks
- Immutable or append-only financial and security audit records
- Dependency, vulnerability, SBOM and licence scanning in CI
- SAST, DAST, integration testing and independent penetration testing
- Device binding, encrypted offline storage and remote device revocation
- Daily reconciliation between financial transactions and operational projections

---

10. Deployment architecture

The initial deployment can use Emeraid's approved server environment:

- Nginx for TLS termination and edge controls (`/` to Odoo, `/api` to FastAPI)
- OpenID Connect identity provider

- Odoo application service
- Fineract application service on a supported JVM (internal; not a staff website)
- Integration/API service (FastAPI, same environment, separate process)
- PostgreSQL cluster with separate logical stores for Odoo, Fineract and the integration store
- Queue/event component for reliable asynchronous processing
- Monitoring, audit log aggregation and alerting
- Encrypted backups and tested restore procedures

---

11. Acceptance evidence

The architecture should be accepted through evidence, not only diagrams:

- Fineract loan and savings requirements mapped to the Functional Matrix
- Demonstrated single-source-of-truth financial posting
- No duplicate repayment when a request is retried
- Odoo and Fineract customer and account IDs linked and traceable, including branch and account officer
- Customer application shows only that customer’s accounts
- Institution isolation through web, mobile, API, reporting and sync
- Unlicensed modules unavailable in the interface and the API
- Reconciliation report with zero unexplained variance for agreed test data
- Cross-system failure and recovery tests
- Offline sync zero-loss and zero-duplication evidence
- PayGo token eligibility, retry, delivery and manual-exception tests
- Load and performance tests on the agreed infrastructure
- Security testing, penetration-test remediation and audit evidence
- Clean deployment, backup and restore demonstration

---

12. Reference basis

- Apache Fineract Platform Documentation — API-oriented architecture, PostgreSQL support, tenant configuration and extension mechanisms
- Apache Fineract Project — project, release and licensing information